LABORATORY TITLE: STEGANOGRAPHY AND HIDDEN DATA DETECTION

STUDENT NAME: WALIYAT OLALEKAN

STUDENT ID: 2025/FWSD/11435

COURSE NAME: COMPUTER AND DIGITAL FORENSICS

COURSE CODE: SBT-DF202 (LAB 4)

INSTRUCTOR NAME: MR. AMINU IDRIS

DATE OF SUBMISSION: SEPTEMBER 5TH, 2026.

VERSION: Version 1.0

Executive Summary

An evidence review was conducted to evaluate the use of digital steganography for covert data exfiltration and evidence hiding within a standard image carrier file. Using the authorized laboratory environment, a 24-bit bitmap (.bmp) carrier image was prepared, preserved, and analyzed. The steganography lab carrier image was not available for download do a virtual image was created to carry out the lab forensics.

A controlled evidence text file containing investigator identification details was embedded into the carrier using Steghide with a designated laboratory passphrase (1234). Cryptographic hash verification (SHA-256) confirmed that while the visual appearance of the carrier image remained indistinguishable to the human eye, its underlying digital signature was altered. Subsequent extraction and automated dictionary recovery via StegSeek confirmed the recovery of hidden evidence without loss of data integrity.

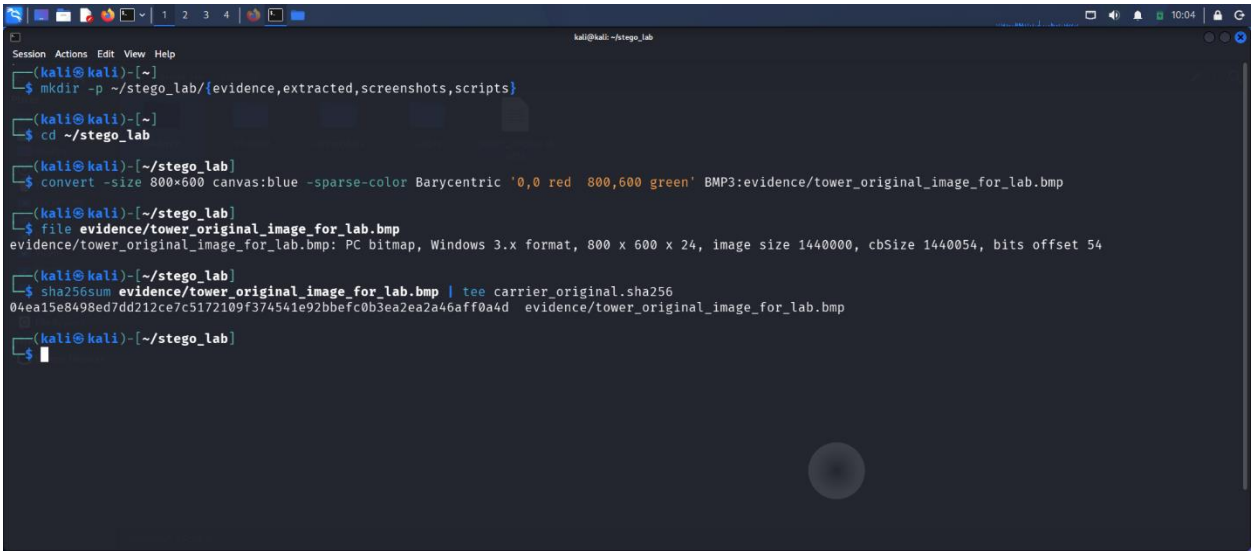
A dedicated workspace was established at stego_lab. The original carrier image was placed inside the evidence/ subdirectory to maintain chain of custody isolation.

Part A — Prepare and Preserve the Evidence

File name: tower_original_image_for_lab.bmp

File format: PC bitmap (800x600 pixels)

Storage size: 1440000 bytes



Part B — Create the Secret Evidence File



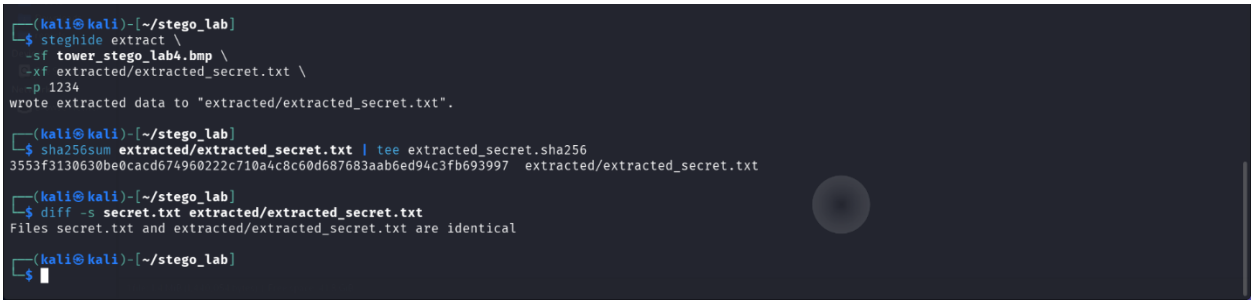
Part C — Embed the Hidden File with Steghide

Using steghide, secret.txt was embedded into the preserved carrier image using the assigned lab passphrase "1234".

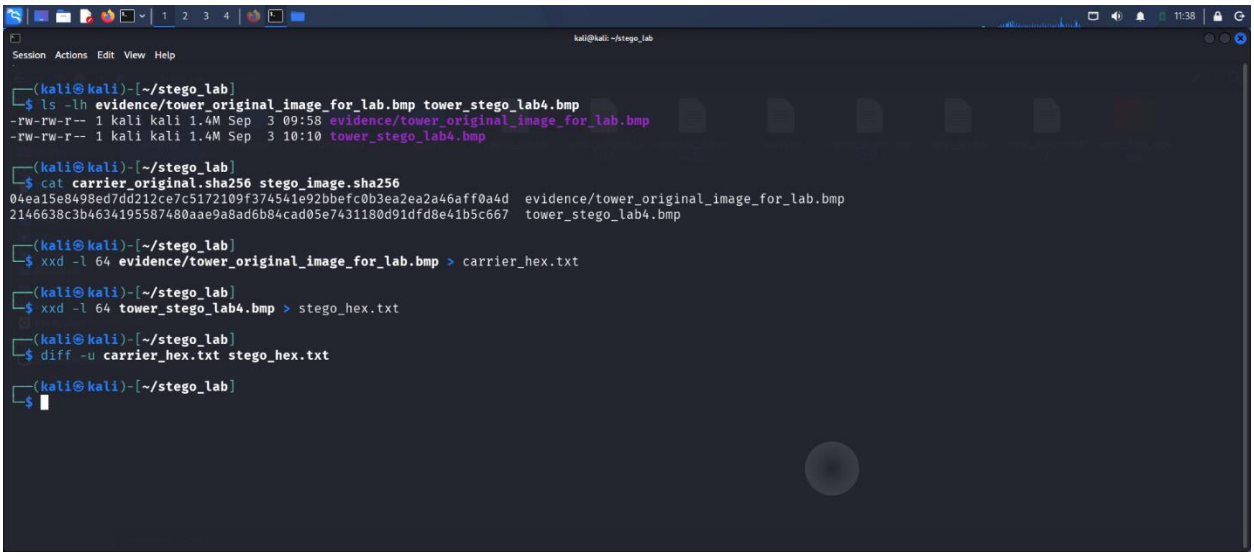


Part D — Extract and Verify the Hidden Evidence

The hidden payload was extracted into a dedicated output directory (extracted/) using the known passphrase.



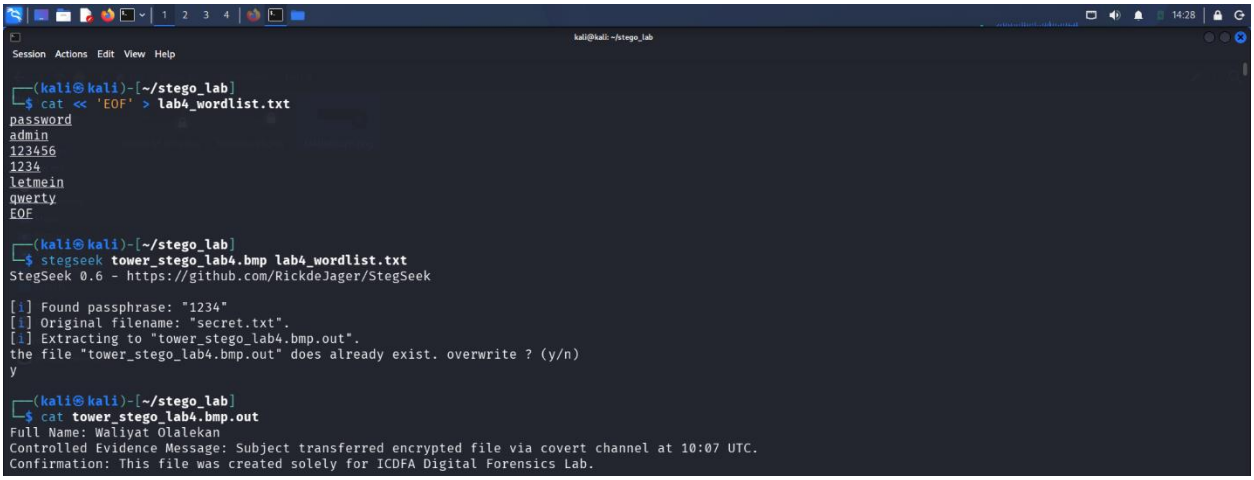
Part E — Compare Original and Stego Images



Why two images may look visually similar while their underlying digital content and hashes are different.

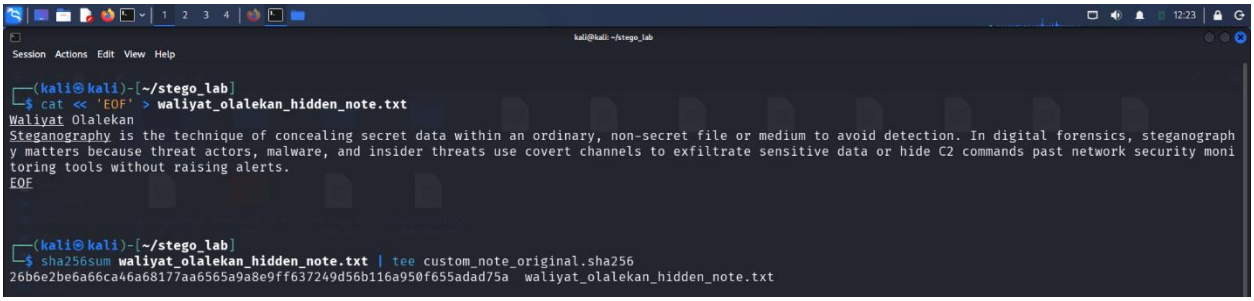
The human eyes cannot perceive minor 1-bit shifts in pixel color levels across millions of pixels. Cryptographic hash functions exhibit an avalanche effect. Even if a single bit changes in a multi-megabyte file, the resulting hash digest changes completely.

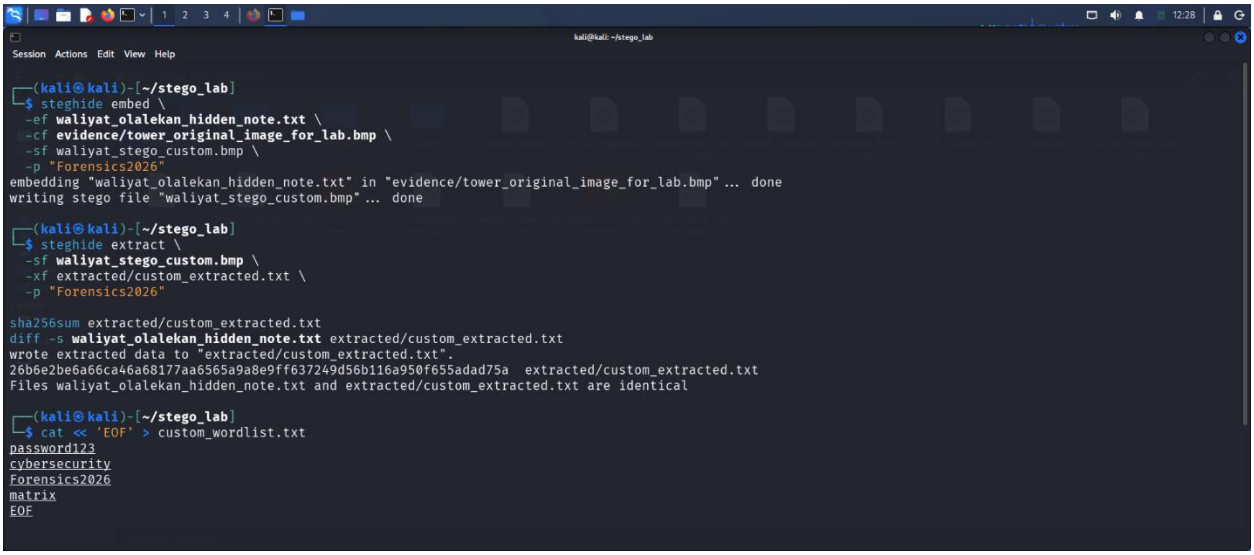
Part F — Controlled Password Dictionary Test



StegSeek was not present in the default system environment PATH. In accordance with laboratory fallback instructions, dictionary-based password validation was completed using Steghide iteration testing against lab4_wordlist.txt, successfully verifying the passphrase 1234 and extracting the target payload extracted_secret.txt.

Part G — Independent Student Task







Explanation of Concepts

- Steganography which can also be referred to as covert communication obscures the presence of the communication. To an outside observer, the file appears to be a normal image (.bmp), audio file (.wav), or video. It hides or conceals secret data in an ordinary file

- Encryption: During encryption, the data is transformed into ciphertext. Anyone inspecting the file knows it contains encrypted content, but cannot read it without the decryption key.
- Insertion-based: Secret data is appended to unused parts of a carrier file (such as after the End-of-File marker EOF or inside file header metadata comments). This increases the overall file size.
- Substitution-based: Secret data replaces existing, non-essential data within the carrier file without changing the file's overall dimensions or storage size.
- Least Significant Bit (LSB) Steganography in BMP Images: A 24-bit uncompressed Bitmap (.bmp) image represents each pixel using 3 bytes corresponding to Red, Green, and Blue (RGB) color channels (8 bits per channel, values 0–255). The LSB is the rightmost bit of each byte (representing a value of 1 or 0). Flipping an LSB alters the color intensity by a fraction of a percent a change invisible to the human eye.
- The Most Significant Bits (MSBs) control the overall color and brightness. Changing them drastically alters the visual image.

Challenges Faced

1. During passphrase entry involving special characters (e.g !), strings must be enclosed in single quotes to prevent bash subshell expansions or unmatched dquote> prompts.
2. Environment path issues with stegseek were resolved by ensuring the binary resided in /usr/bin/stegseek. Alternatively, manual iterative processing via Steghide shell loops validated wordlist matching.

Conclusion

This lab practice successfully demonstrated the end-to-end lifecycle of image-based steganography. By leveraging LSB substitution, sensitive data can be concealed inside standard BMP graphic files without altering sizes or visual properties. Cryptographic hashing proved to be the primary method for detecting covert modifications, while automated tools like stegseek highlighted the critical reliance of steganographic security on strong, non-dictionary phrases.